



INTELIGENCIA ARTIFICIAL EJERCICIO 1

MÁS CAPAS EN EL PERCEPTRÓN MULTICAPA (IRIS)

**CARBALLO ROSAS XAIL DEL CARMEN
LIC. EN INGENIERÍA EN COMPUTACIÓN**

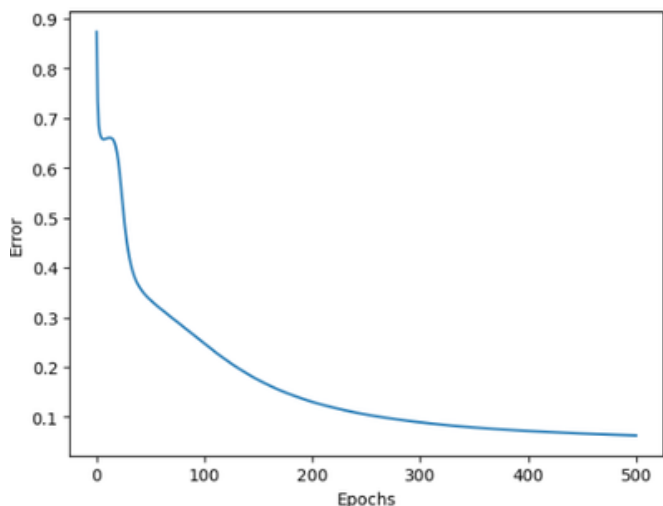
PROFESOR: VICTOR EMANUEL UC CETINA

SEPTIEMBRE 30 DEL 2026

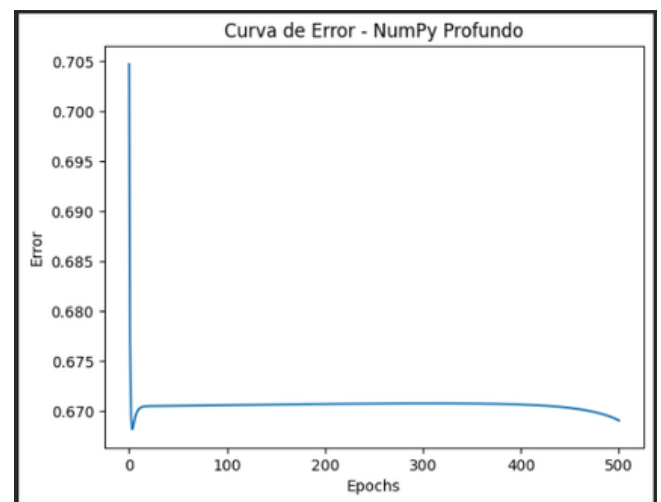
EJERCICIO 1 — MÁS CAPAS EN EL PERCEPTRÓN MULTICAPA (IRIS)

Evidencias:

- Original: Multilayer perceptron



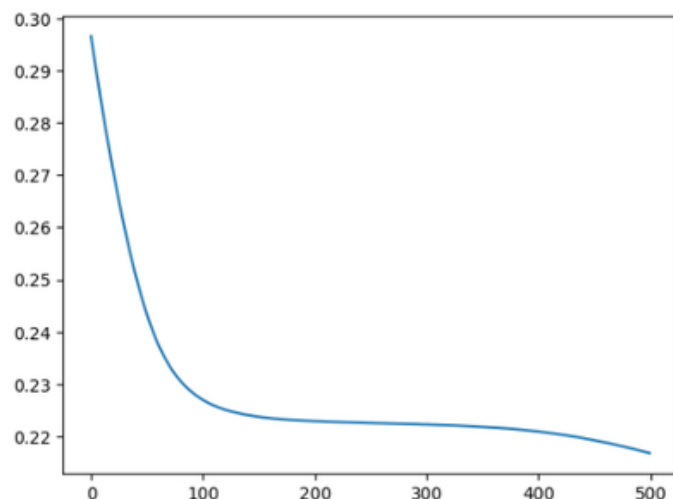
- Profundo: Multilayer perceptron



- Original: Keras

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

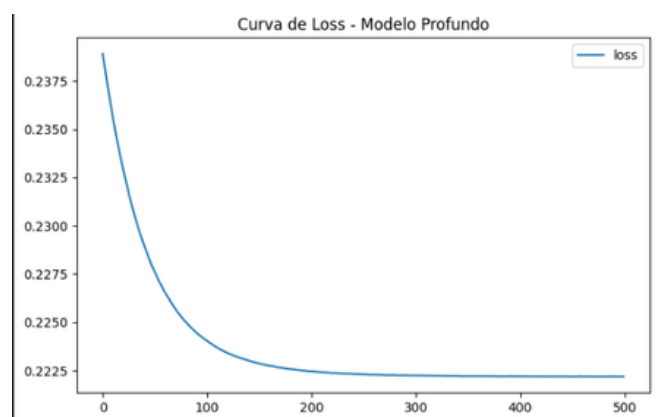
Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)



- Profundo: Keras

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)



Implementacion	Topologia	Error Final	Tiempo (S)
Multilayer	4 x 3 x 3	0.05531612331972256	N/A
Multilayer	4 x 3 x 3 x 3 x 3	0.3000	10.21
Keras	4 x 3 x 3	0.039	N/A
Keras	4 x 3 x 3 x 3 x 3	0.2221	25.61

- **¿Bajó más el error al añadir dos capas, o se estancó / empeoró? ¿Igual en NumPy y en Keras?**

El error no disminuyó al añadir capas; por el contrario, el modelo empeoró y el aprendizaje se estancó. En Keras, la pérdida final empeoró de **0.039** en la topología original a **0.2223** en la profunda.

En la implementación de NumPy, el modelo profundo también se deterioró pues pasó a tener un error de 0.3 cuando inicialmente tenía solo 0.055. *Este deterioro ocurrió tanto en Keras como en NumPy.*

- **¿Las curvas de la notebook 01 y de Keras se parecen con la misma topología? Si no, ¿qué diferencias de implementación podrían explicarlo?**

Las curvas tienen una forma similar con una caída asintótica, pero sus trayectorias no son idénticas. La gráfica de Keras muestra una caída más controlada y suave desde el inicio. Esta diferencia se explica por la implementación :

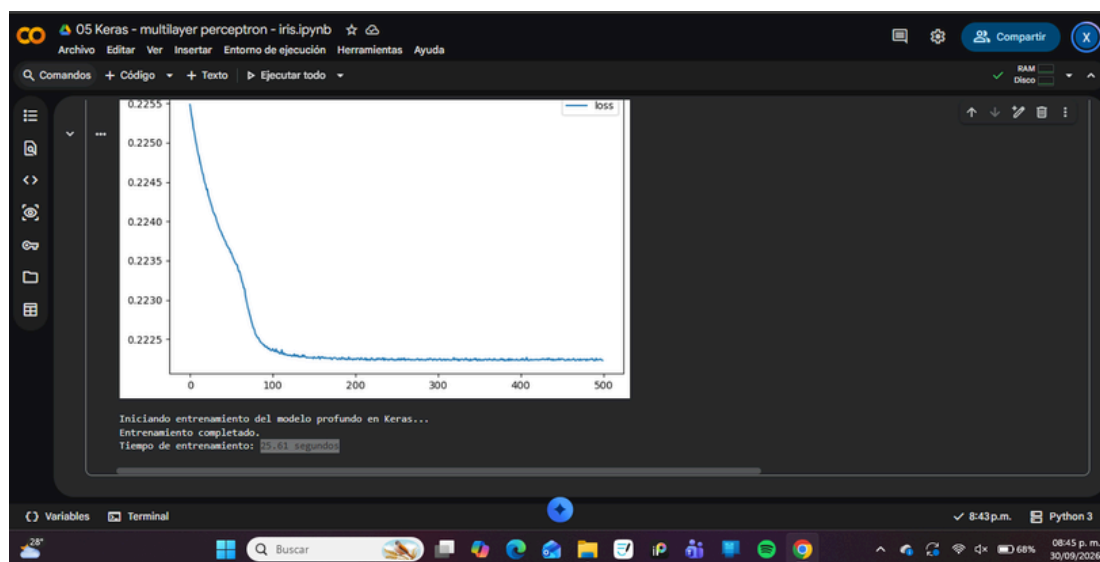
Keras utiliza algoritmos de inicialización de pesos más sofisticados y realiza cálculos con mayor eficiencia, la versión de **NumPy** depende de una inicialización aleatoria más simple y procesa los deltas mediante iteraciones secuenciales con bucles for, lo que genera variaciones en la ruta de minimización del error

- Con sigmoides apiladas y MSE, ¿tiene sentido que una red más profunda no aprenda mejor en Iris?

Sí, debido al desvanecimiento del gradiente, la derivada de la función sigmoide produce valores máximos de 0.25.

Durante el proceso de retropropagación, multiplicar repetidamente valores menores a 1 a través de cuatro capas consecutivas provoca que los gradientes de error (δ) lleguen microscópicos a las primeras capas. Entonces, los pesos iniciales apenas se actualizan y el aprendizaje se congela. Además de esto, el conjunto de datos Iris es simple; forzar una arquitectura profunda $4 \times 3 \times 3 \times 3$ introduce una complejidad injustificadamente, haciendo que se perjudique la convergencia en lugar de ayudarla.

EVIDENCIAS DE EJECUCION EN COLAB



04 Multilayer perceptron.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todo

RAM Disco

```
#Tiempo de la celda de entrenamiento
start_time = time.time()

for e in range(epochs):
    for d in range(len(X)):
        # FORWARD PROPAGATION
        output_layer1 = np.zeros(num_neurons_layer1)
        for i in range(len(output_layer1)):
            output_layer1[i] = propagate(X[d], layer1[i])

        output_layer2 = np.zeros(num_neurons_layer2)
        for i in range(len(output_layer2)):
            output_layer2[i] = propagate(output_layer1, layer2[i])

        output_layer3 = np.zeros(num_neurons_layer3)
        for i in range(len(output_layer3)):
            output_layer3[i] = propagate(output_layer2, layer3[i])

        output_layer4 = np.zeros(num_neurons_layer4)
        for i in range(len(output_layer4)):
            output_layer4[i] = propagate(output_layer3, layer4[i])

        # retropropagacion

        # Delta de Capa 4
        delta_layer4 = np.zeros(num_neurons_layer4)
        for i in range(len(delta_layer4)):
            delta_layer4[i] = output_layer4[i] * (1 - output_layer4[i]) * (Y[d][i] - output_layer4[i])
```

Variables Terminal

Ejecutando (4 s) Python 3